SIMATS Engineering

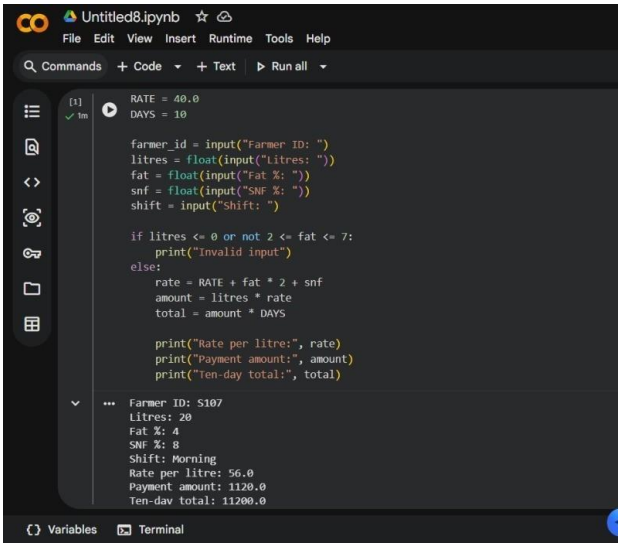
Name	Sharmila S	
Register Number	1912260141	
Department	B.Tech ECE	
Course Name	Python Programming	
Course Code	CSA0823	
Slot	C	
GitHub link	https://github.com/ssharmila8125-netizen/sharmisCSA08-1912260141	

Program 1: Milk Payment Calculation with Input Validation

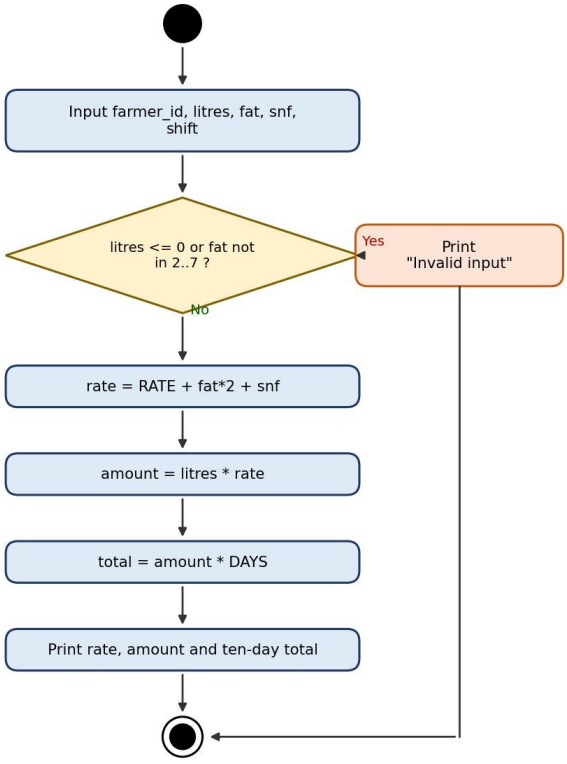
Code

```
RATE = 40.0
DAYS = 10
 farmer_id = input("Farmer ID: ")
litres = float(input("Litres: "))
fat = float(input("Fat %: ")) snf =
float(input("SNF %: ")) shift =
input("Shift: ")  if litres <= 0 or
not 2 <= fat <= 7:
    print("Invalid input")
else:
    rate = RATE + fat * 2 + snf
amount = litres * rate     total =
amount * DAYS      print("Rate per
litre:", rate)     print("Payment
amount:", amount)     print("Ten-
day total:", total)
```

Output Screenshot



UML Activity Diagram

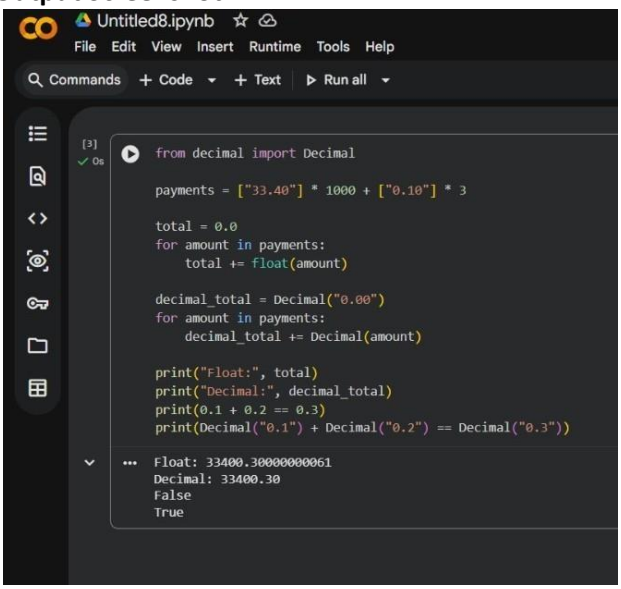


Program 2: Float vs Decimal Precision in Payments

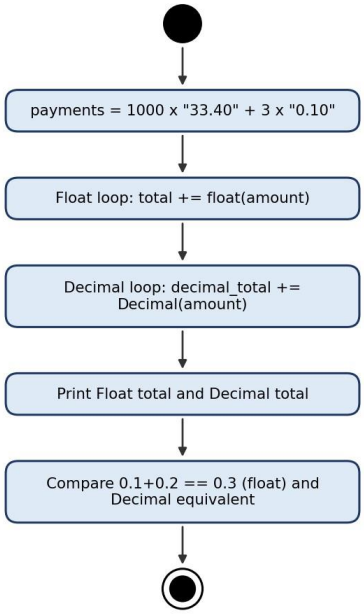
Code

```
from decimal import Decimal
payments = ["33.40"] * 1000 + ["0.10"] * 3
total = 0.0
for amount in payments:
    total += float(amount)
decimal_total = Decimal("0.00")
for amount in payments:
    decimal_total += Decimal(amount)
print("Float:", total)
print("Decimal:", decimal_total)
print(0.1 + 0.2 == 0.3)
print(Decimal("0.1") + Decimal("0.2") ==
Decimal("0.3"))
```

Output Screenshot



UML Activity Diagram



Program 3: Fat/SNF Steps, Rate Calculation and Swapping Values

Code

```
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
fat = 4.0
snf = 8.0
fat_steps = fat * 10
snf_steps = snf * 10
rate = BASE_RATE + fat * FAT_RATE + snf * SNF_RATE
amount = 10 * rate
print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
```

```
print("Rate per litre:", rate)
print("Payment amount:", amount)
a = 10
b = 20
a, b = b, a
print("Swapped:", a, b)
```

Output Screenshot **UML Activity Diagram**

```
[4]
✓ 0s
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30

fat = 4.0
snf = 8.0

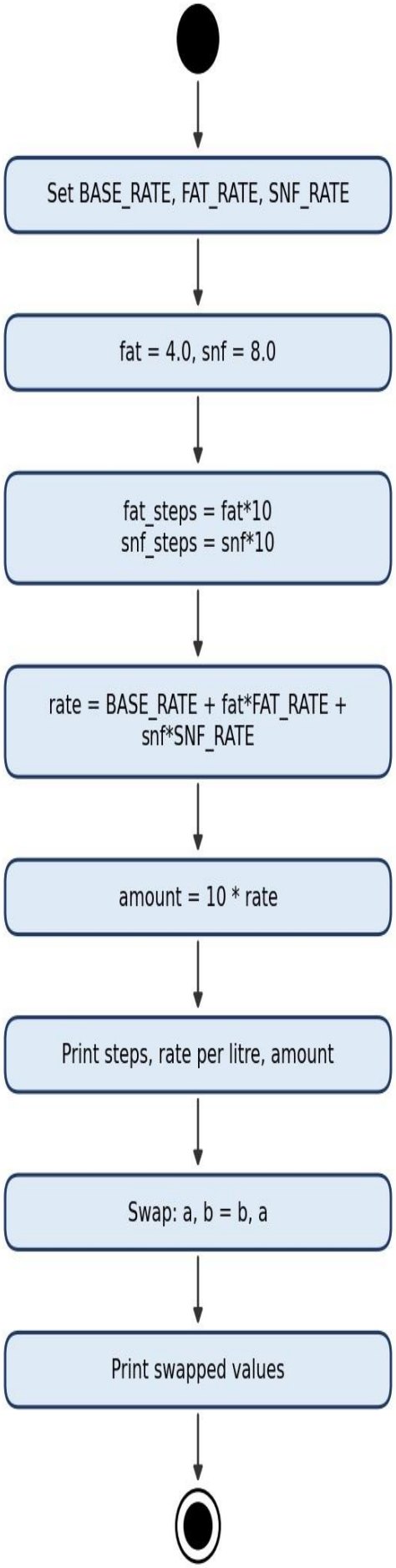
fat_steps = fat * 10
snf_steps = snf * 10

rate = BASE_RATE + fat * FAT_RATE + snf * SNF_RATE
amount = 10 * rate

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate per litre:", rate)
print("Payment amount:", amount)

a = 10
b = 20
a, b = b, a
```

```
Fat steps: 40.0
SNF steps: 80.0
Rate per litre: 34.0
Payment amount: 340.0
Swapped: 20 10
```



Program 4: Step Calculation and Floating-Point Rounding Issues

Code

```
Fat = 4.2

Snf = 3.1

Fat_steps = round((fat – 3.5) / 0.1)

Snf_steps = round((snf – 3.0) / 0.1)



Rate = 30.0 + fat_steps * 0.40 + snf_steps * 0.30

Amount = 10 * rate



Print(“Fat steps:”, fat_steps)

Print(“SNF steps:”, snf_steps)

Print(“Rate per litre:”, rate)

Print(“Amount:”, amount)



Print((4.2 – 3.5) / 0.1)

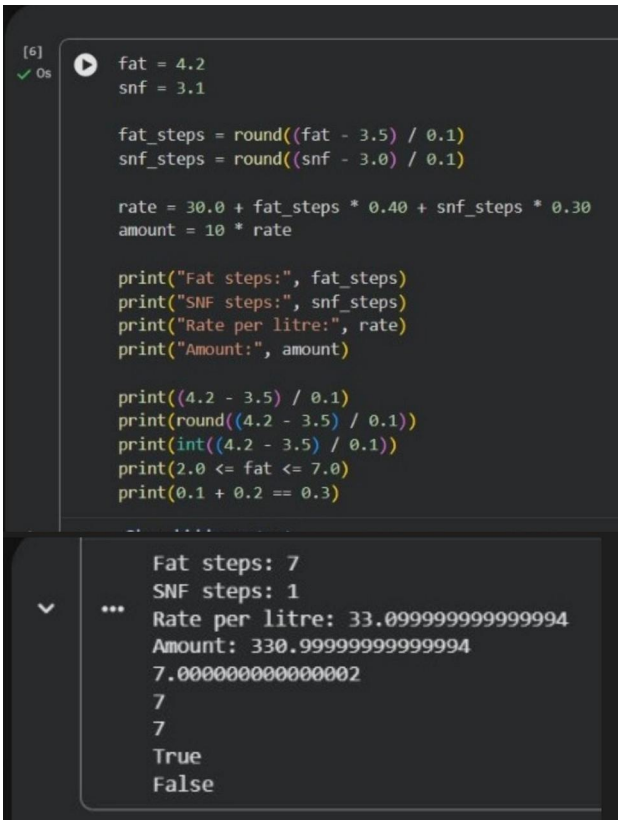
Print(round((4.2 – 3.5) / 0.1))

Print(int((4.2 – 3.5) / 0.1))

Print(2.0 <= fat <= 7.0)

Print(0.1 + 0.2 == 0.3)
```

Output Screenshot



```
[6]
✓ 0s
fat = 4.2
snf = 3.1

fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 3.0) / 0.1)

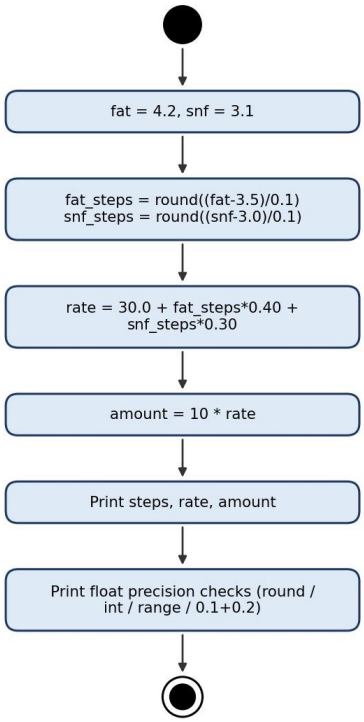
rate = 30.0 + fat_steps * 0.40 + snf_steps * 0.30
amount = 10 * rate

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate per litre:", rate)
print("Amount:", amount)

print((4.2 - 3.5) / 0.1)
print(round((4.2 - 3.5) / 0.1))
print(int((4.2 - 3.5) / 0.1))
print(2.0 <= fat <= 7.0)
print(0.1 + 0.2 == 0.3)

Fat steps: 7
SNF steps: 1
Rate per litre: 33.099999999999994
Amount: 330.99999999999994
7.000000000000002
7
7
True
False
```

UML Activity Diagram

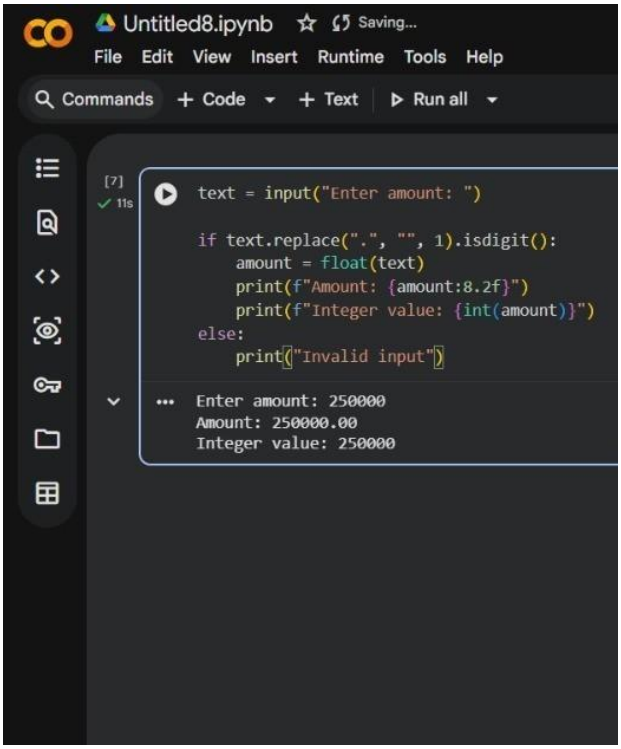


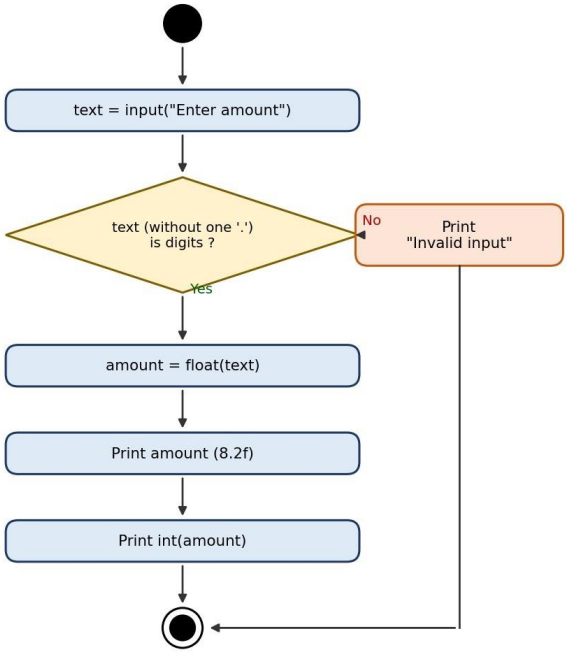
Program 5: Amount Input Validation and Formatting

Code

```
text = input("Enter amount: ")
if text.replace(".", "", 1).isdigit():
    amount = float(text)
    print(f"Amount: {amount:8.2f}")
    print(f"Integer value: {int(amount)}")
else:
   print("Invalid input")
```

Output Screenshot **UML Activity Diagram**



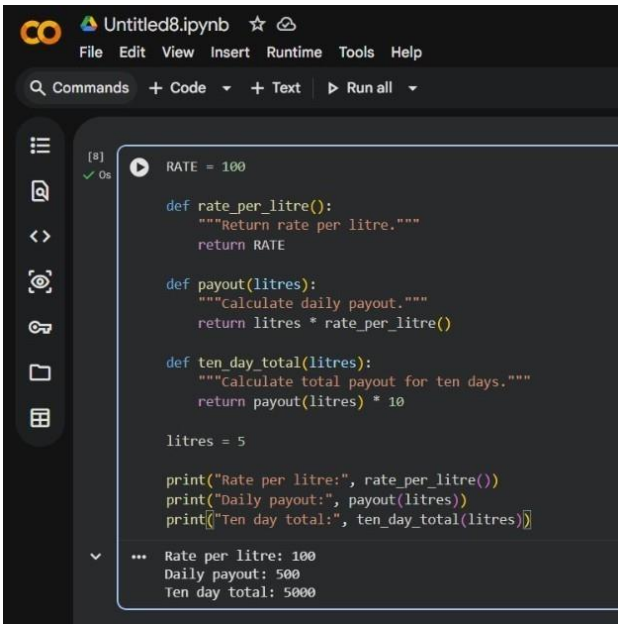


Program 6: Rate and Payout using Functions

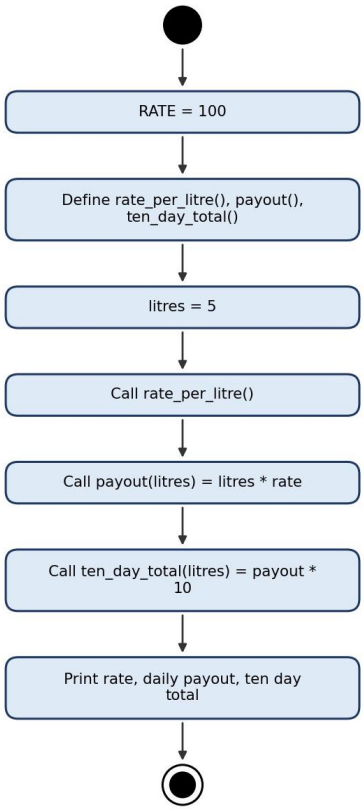
Code

```
RATE = 100
Def rate_per_litre():
    “””Return rate per litre.”””
    Return RATE
Def payout(litres):
    “””Calculate daily payout.”””
    Return litres * rate_per_litre()
Def ten_day_total(litres):
    “””Calculate total payout for ten days.”””
    Return payout(litres) * 10
Litres = 5
Print(“Rate per litre:”, rate_per_litre())
Print(“Daily payout:”, payout(litres))
Print(“Ten day total:”, ten_day_total(litres))
```

Output Screenshot



UML Activity Diagram

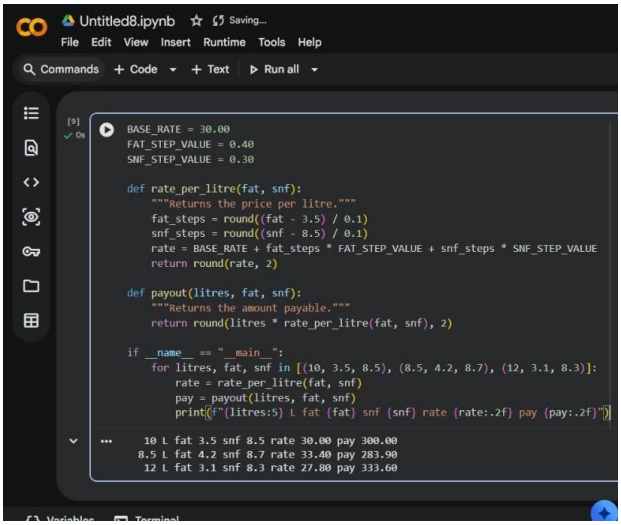


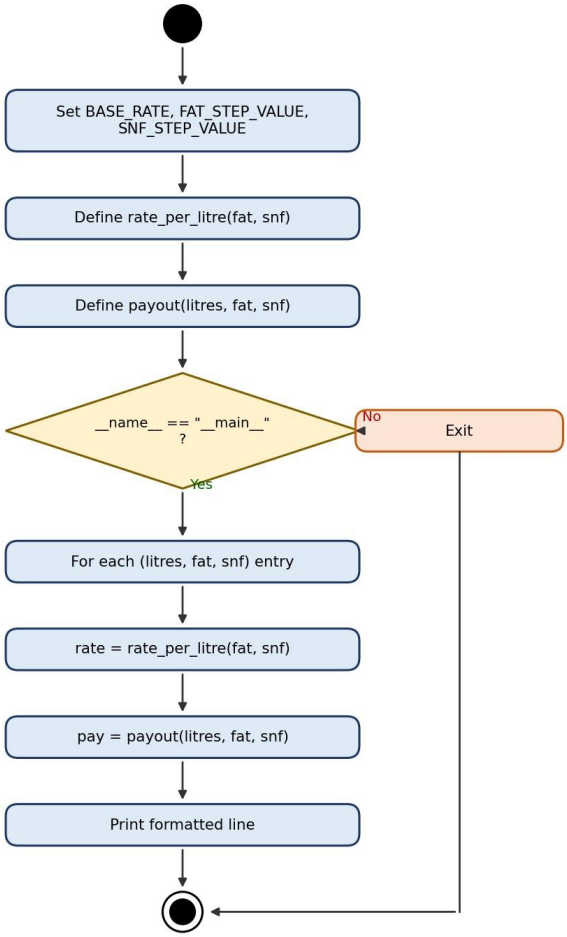
Program 7: Rate per Litre Function with Multiple Entries

Code

```
RATE = 100
def rate_per_litre():
    """Return rate per litre."""
    return RATE
def payout(litres):
    """Calculate daily payout."""
    return litres * rate_per_litre()
def ten_day_total(litres):
    """Calculate total payout for ten days."""
    return payout(litres) * 10
litres = 5
print("Rate per litre:", rate_per_litre())
print("Daily payout:", payout(litres))
print("Ten day total:", ten_day_total(litres))
```

Output Screenshot **UML Activity Diagram**



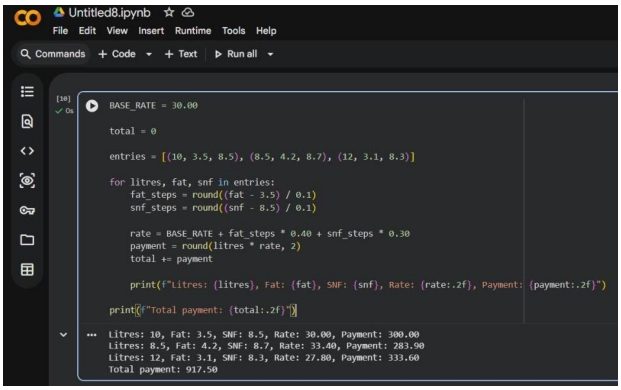


Program 8: Total Payment for Multiple Entries

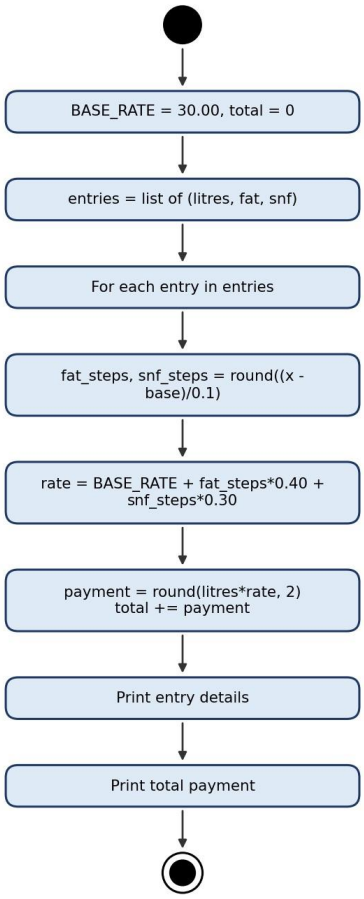
Code

```
BASE_RATE = 30.00
total = 0
entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
for litres, fat, snf in entries:
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * 0.40 + snf_steps * 0.30
    payment = round(litres * rate, 2)
    total += payment
    print(f"Litres: {litres}, Fat: {fat}, SNF: {snf}, Rate: {rate:.2f}, Payment: {payment:.2f}")
print(f"Total payment: {total:.2f}")
```

Output Screenshot



UML Activity Diagram



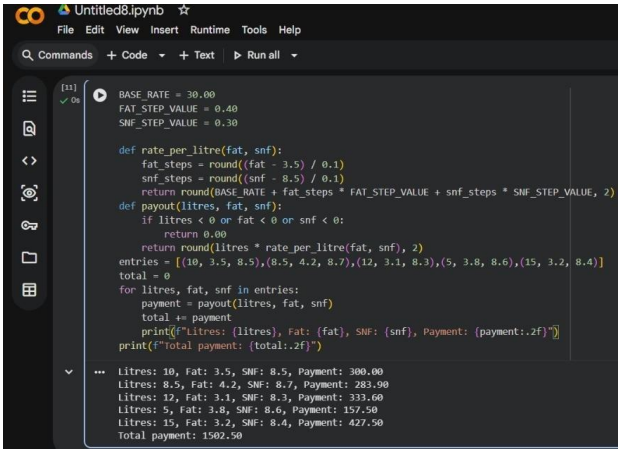
Program 9: Payout with Validation and Total Payment

Code

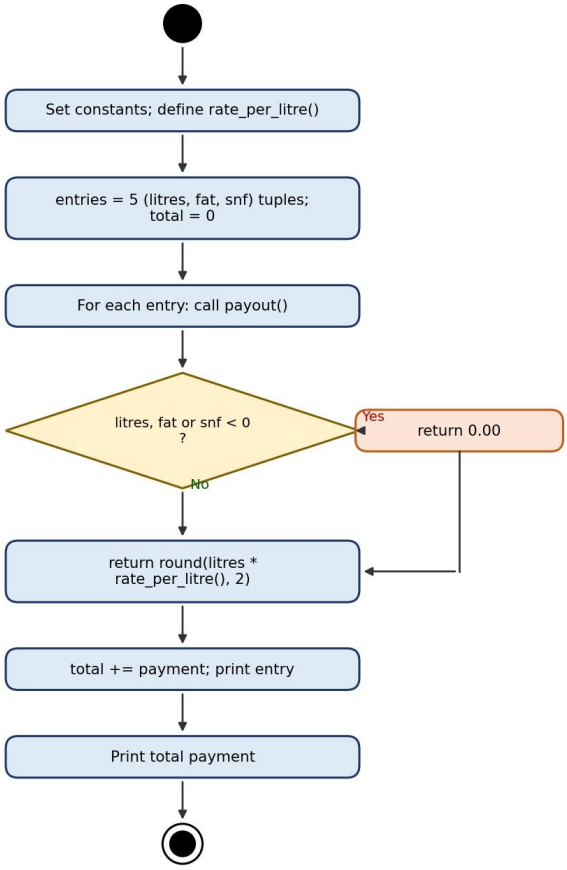
```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
def payout(litres, fat, snf):
    if litres < 0 or fat < 0 or snf < 0:
        return 0.00
    return round(litres * rate_per_litre(fat, snf), 2)
entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7),(12, 3.1, 8.3),(5, 3.8, 8.6),(15, 3.2, 8.4)]
total = 0
for litres, fat, snf in entries:
    payment = payout(litres, fat, snf)
    total += payment
    print(f"Litres: {litres}, Fat: {fat}, SNF: {snf}, Payment: {payment:.2f}")

print(f"Total payment: {total:.2f}")
```

Output Screenshot



UML Activity Diagram

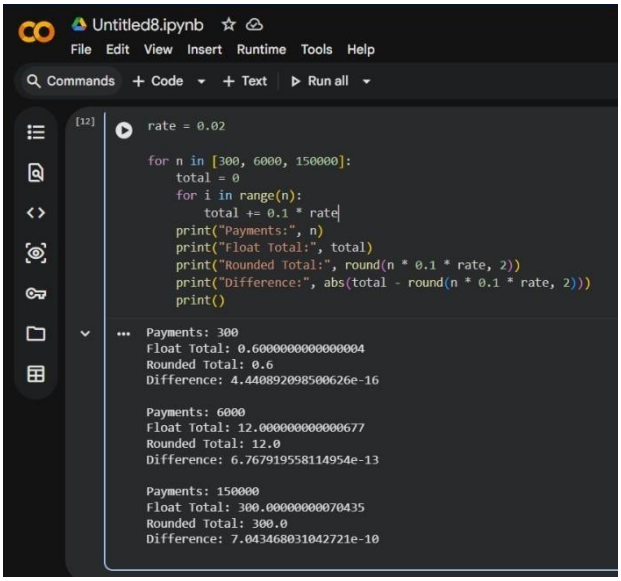


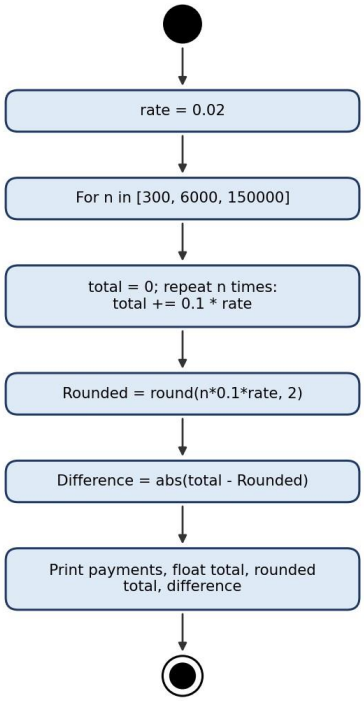
Program 10: Floating-Point Accumulation Error Analysis

Code

```
rate = 0.02
for n in [300, 6000,
150000]:
    total = 0
    for i in range(n):
        total += 0.1 * rate
    print("Payments:", n)
    print("Float Total:",
total)
    print("Rounded Total:",
round(n * 0.1 * rate, 2))
    print("Difference:",
abs(total - round(n * 0.1 *
rate, 2)))
    print()
```

Output Screenshot **UML Activity Diagram**





Program 11: Milk Amount Calculation and Design Notes

Code

```
Fat_rate = 0.05


Snf_rate = 0.03


Fat = 4.50


Snf = 8.50


Amount = (fat * fat_rate) + (snf * snf_rate)
```

```
Print("Fat:", round(fat, 2))

Print("SNF:", round(snf, 2))

Print("Milk Amount:", round(amount, 2))

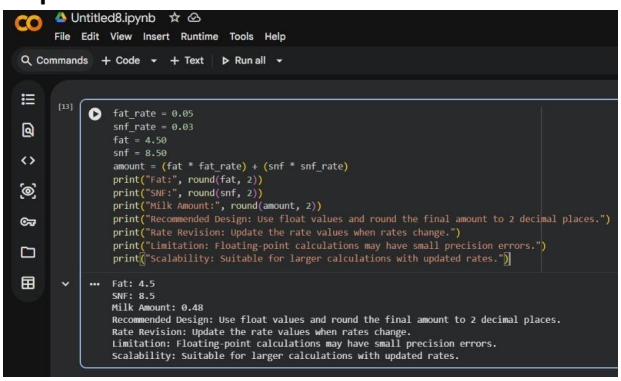
Print("Recommended Design: Use float values and round the final amount to 2 decimal places.")

Print("Rate Revision: Update the rate values when rates change.")

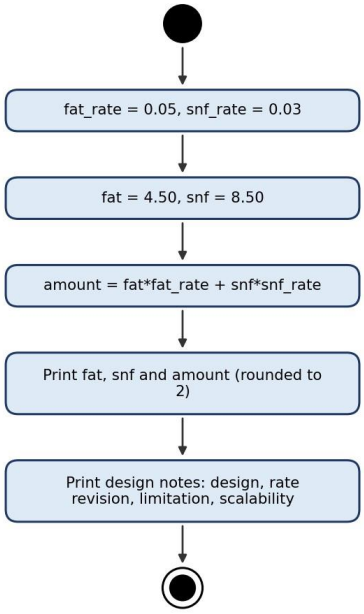
Print("Limitation: Floating-point calculations may have small precision errors.")

Print("Scalability: Suitable for larger calculations with updated rates.")
```

Output Screenshot



UML Activity Diagram

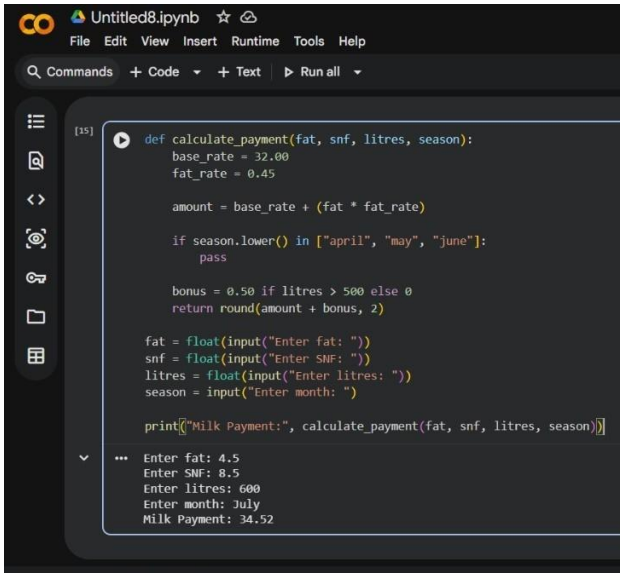


Program 12: Milk Payment with Month Bonus Function

Code

```
def calculate_payment(fat, snf, litres, season):
    base_rate = 32.00
    fat_rate = 0.45
    amount = base_rate + (fat * fat_rate)
  if season.lower() in ["april", "may", "june"]:
        pass
    bonus = 0.50 if litres > 500 else 0
    return round(amount + bonus, 2)
fat = float(input("Enter fat: "))
snf = float(input("Enter SNF: "))
litres = float(input("Enter litres: "))
season = input("Enter month: ")
print("Milk Payment:", calculate_payment(fat, snf, litres, season))
```

Output Screenshot



UML Activity Diagram

